Gossip Protocol Modular Pseudocode

Concepts explanations and a modular reference design

Mark Ray-Smith Cityline Ltd

Revised 17 October 2026 | Basic programming knowledge assumed

What this document provides

Gossip lets running programs spread information through repeated exchanges with a few other programs. Each numbered section begins by defining the words in its title and explaining the functionality in ordinary language. A concrete example shows what that functionality accomplishes. The explanations, step walkthroughs and pseudocode then appear together within the same section.

There is no single universal gossip wire protocol. The design below combines established techniques and makes its own explicit choices. It is an educational specification, not a wire-compatible implementation of SWIM, memberlist, Cassandra or Dynamo. All pseudocode is original synthesis; research citations identify the concepts that informed it.

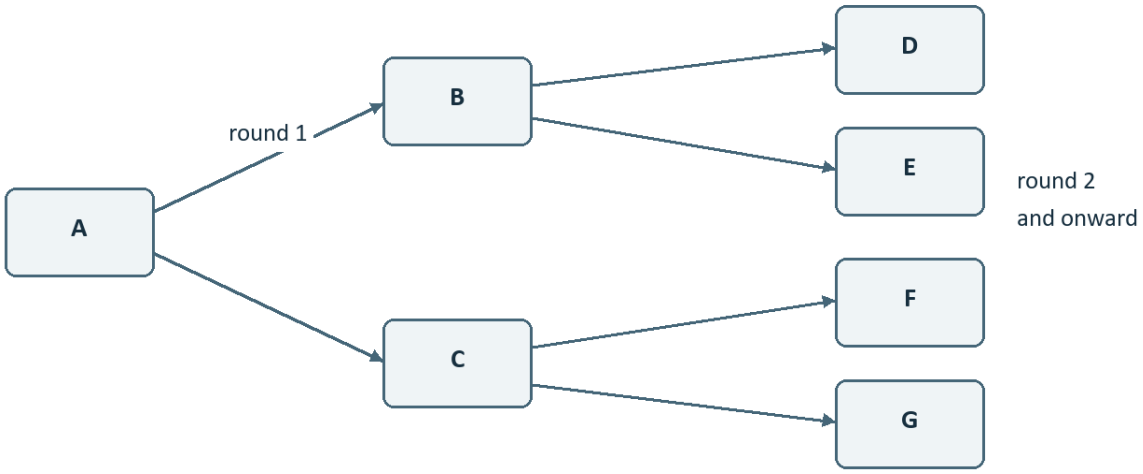


Figure 1 A new update spreads through a few peer contacts per round. Real contacts overlap and messages can be lost; this illustration is not a delivery guarantee.

The central design rule

Keep membership, failure detection and application replication separate. Membership says which process instances are known. Failure detection estimates reachability. Replication reconciles application state. None of these alone establishes consensus, a unique leader, or exactly once execution.

The reference system replicates a bounded key-value namespace to every admitted node. It uses best effort datagrams for fast spreading and authenticated reliable streams for bootstrap and repair. A write succeeds locally after durable commit; cluster-wide delivery is asynchronous.

Reading route

The introductory chapters explain the setting and notation. Numbered sections 1 to 14 describe the protocol modules, each beginning with its own functional explanation. Sections 15 to 18 cover examples, guarantees, operations and validation. Sections 19 and 20 provide sources and security contracts. The glossary is a lookup aid. Word's navigation pane lists all chapters.

Getting started with nodes and local views

A node is a running program taking part in the system. A local view is the information that one node currently holds about the other programs and their data. This chapter explains why these views can differ and what the protocol does to bring them together.

A node is one running participant

Imagine three server programs called A, B and C. Each program is a node. Together they form a cluster. Each node can send messages to other nodes, called its peers. An endpoint is the address and port at which a node receives messages. The same physical computer can run more than one node.

A distributed system has no shared memory across these programs. A cannot inspect B's variables. It learns about B by receiving messages. Messages take time, so what A knows can differ from what B or C knows at the same moment. Each node therefore maintains its own local view of the cluster and its data.

Two different kinds of local knowledge

The membership table records process instances, their addresses and their locally known status. The application store records business or service data, such as a configuration key called colour. Learning that B is reachable does not tell A whether B has the latest colour value. The protocol maintains these two forms of knowledge separately.

Node	Membership view	Local colour value
A	A and B known alive	blue at version 8
B	A B and C known alive	blue at version 8
C	B and C known alive	red at version 5

This table is an ordinary temporary condition. B knows about C before A does. C has not yet received the latest colour update. Later exchanges can bring their views together. A membership entry is a local belief; it is not a live connection to another process.

What convergence means

Convergence means replicas eventually reach the same result after they receive the relevant information and apply the same merge rules. A replica is a node holding a copy of data. A merge rule decides how an incoming record changes the local copy. Eventual consistency allows temporary differences while updates travel.

For example, if no one writes a newer colour and messages can flow again, C can learn version 8 and return blue. There is no promise that every node returns blue immediately after A writes it. A successful local write means A committed its copy; it says nothing yet about C.

The example used throughout

Follow A, B and C as they share colour updates, check reachability, and recover after missed messages. The later pseudocode adds identifiers, versions and limits to make those simple interactions safe to repeat.

Getting started with messages and gossip

A message is information sent from one running program to another. Gossip is a way of spreading information through repeated exchanges with a few other programs at a time. This chapter follows one update through the fast exchanges and the later checks that recover missed updates.

What is being exchanged

A record is one piece of stored information: for example, colour is blue at version 8, or instance B is suspected at incarnation 4. A message is a temporary package sent between nodes. It can carry a request, a reply and several records. An envelope is the outer message structure containing sender, cluster, message kind and authentication information.

A rumor is a record waiting to be spread. In this document it is not an unverified human claim: it is a candidate protocol update that must pass validation. A gossip round is one scheduled opportunity to send records to selected peers. Nodes have their own schedules; there is no global round clock.

Push pull and piggybacking

Term	Basic action	Example
Push	Send information without waiting for a request	A sends its new colour to B
Pull	Ask another node for information	C asks B for a missing record
Push pull	Exchange information in both directions	A and B compare their stored versions
Piggyback	Attach updates to an existing message	A adds a colour rumor to a PING

Fanout is the maximum number of peers contacted in one action. With fanout 2, A sends an update to two selected peers rather than every peer. Those peers can spread it again. Random selection helps different parts of the cluster hear the information; repeated contacts also reach peers that missed an earlier message.

Why two delivery paths are needed

The fast path uses small datagrams, similar to UDP packets. A datagram is one independently sent message; the network can lose it, deliver it twice or deliver a later one first. Repeated gossip makes progress despite some losses, but a fixed retry budget can still miss a node.

The repair path periodically compares retained state over a reliable ordered stream, similar to a TCP connection with authentication. This comparison is called anti entropy: it reduces differences between replicas. The stream may still disconnect, and successful transfer does not mean the receiver committed a record until the application confirms that step.

A complete miniature exchange

A stores blue at version 8, then pushes it to B. B merges it and adds it to its own rumor queue. C misses all fast messages. Later C compares its inventory with B and pulls version 8. Repeated spreading provides speed; retained state and repair provide another opportunity after the fast attempts stop.

Getting started with failures and versions

Failure detection estimates whether another program is available by checking whether it replies. Versions are labels used to distinguish newer information from older information. The protocol needs both: it must react to silence without allowing delayed messages to undo a newer decision.

Silence does not prove a crash

A probe asks whether a peer can respond. PING is the request and ACK, short for acknowledgment, is the reply. A timeout is a local decision that the reply has taken too long. A missed reply may mean a crashed process, a lost packet, a busy receiver or a broken network path. A partition is a network split that prevents some otherwise running nodes from communicating.

The failure detector first tries a direct probe, then asks helpers to try alternate paths. If every attempt fails, the target becomes SUSPECT. That means the local node has reason to doubt reachability. After a grace interval it can become DEAD, meaning locally considered unavailable. Neither label is an objective proof that the process has stopped. LEFT means the owner deliberately announced its departure.

Two version systems with different jobs

Version	What it belongs to	Why it is needed
Incarnation number	An owner announcement for one process instance	Lets the owner refute old suspicion
Value stamp	An application write to a key	Orders competing stored values deterministically

An instance identifier names one process lifetime. A restart gets a new identifier. Within one lifetime, an incarnation number versions the owner's announcements. If B hears SUSPECT at incarnation 4, it can announce ALIVE at 5. Old suspicion at 4 then loses. Refutation means publishing this newer evidence about oneself.

A value stamp contains a logical counter and a writer identifier. A logical counter is an integer that advances when the node creates a write; it does not measure seconds. Stamps (8,A) and (9,B) compare first by counter. Equal counters compare by writer identifier, using the same byte ordering at every node. Later chapters show why this produces a repeatable winner.

Deletion must also travel

A tombstone is a stored record saying a key was deleted, with its own stamp. Removing a key locally without keeping that record would leave nothing to tell C, which may still hold an old value. Retaining the newer tombstone lets C learn the deletion and reject an older value arriving afterward. A genuinely newer write can deliberately recreate the key.

Membership incarnations and value stamps must never be interchanged. An ACK can establish a successful probe without proving that the peer has fresh application data.

How to read the pseudocode notation

Pseudocode describes the steps of an algorithm using readable programming notation without requiring a particular programming language. An algorithm is a procedure for carrying out a task. This chapter explains how to read the symbols, data fields and control flow used in the remaining sections.

The notation uses indentation to group steps. Typed records describe information; event handlers describe responses to messages or timer deadlines. It is explanatory notation, not executable Python.

Notation	Read it as
type and enum	A data shape and a fixed set of named choices
r.owner.incarnation	The incarnation field inside the owner inside record r
= == != <=	Assign a value; test equality; test inequality; test less than or equal
[ ] { } and (a b)	Lists or key lookup; sets or record fields; an ordered tuple of fields
max ceil and modulo	Choose the largest; round upward to an integer; take division remainder
map.get(key) and NONE	Look up a key; NONE means no entry is present
require condition	Continue only when the condition is true; otherwise reject
return and continue	End this procedure; or skip to the next loop item
onSomething and Tick	An event handler; or a periodic scheduled event
transaction	Commit the grouped storage changes together or none of them
try catch finally	Attempt work; handle failure; always perform cleanup
# comment	An explanation for the reader, not a protocol action

An annotated small example

Pseudocode

```
mergeExample(incoming):
  require incoming is valid
  old = values.get(incoming.key)
  if old != NONE and incoming.version <= old.version:
    return UNCHANGED
  values[incoming.key] = incoming
  return CHANGED
```

First validate the incoming record. Next look up the local record for that key. If there is a local record and the incoming version is no newer, stop without changing anything. Otherwise store the incoming record and report a change. The indentation places return UNCHANGED inside the if condition; the assignment occurs only when that early return did not happen.

Here incoming is an input and values is state retained by the node. The full algorithms add signatures, tie breaking, durable transactions and dissemination. The miniature example only teaches the control flow.

Abstract helper operations

A helper such as verifyEnvelope has a stated responsibility without selecting a network library or database. Its contract says what it must accomplish. Each section explains the operations before showing their decision order in pseudocode.

1 Scope and research foundations

Scope means the functionality covered by this design. Research foundations are the published ideas used to build it. Here, gossip means that programs repeatedly share information with a small selection of other programs so that it can spread through a larger group.

The design covers joining the group, checking whether other programs respond, spreading recent changes, and recovering information missed during disconnection. It also specifies how competing records are compared. For example, a program that missed a configuration change can obtain it later from a peer. It does not make gossip responsible for choosing a unique leader or authorizing exclusive work.

Three related uses of gossip

Technique	Purpose	Reference behavior
Rumor dissemination	Spread recent changes quickly	Bounded repeated push with deduplication
Anti entropy	Repair missed or older state	Periodic bidirectional snapshot comparison
Membership gossip	Discover joins leaves and suspected failures	SWIM inspired probes plus versioned member records

Demers and colleagues distinguish repeated state reconciliation from spreading individual updates [1]. SWIM separates randomized probing from membership dissemination [2]. memberlist adds practical extensions [3]. These ideas supply the decomposition, while the precise interfaces and algorithms in this document are reference-design decisions.

Assumptions

Peers may crash, restart, pause or become partitioned. Messages may be lost, duplicated or reordered. Local timers use a monotonic clock. Eventually, enough healthy peers must reconnect and exchange messages for reconciliation to make progress. Nodes are authenticated, admitted, and cooperative; Byzantine behavior is outside the model.

Every process start receives a new random instance identifier. A principal identifies an authorized machine or service; an instance identifies one lifetime of a process. Restarts do not reuse old instance identifiers. This avoids conflating old messages with a replacement process.

Explicit choices

Application conflicts use a logical-counter register with a deterministic tie breaker. No wall-clock synchronization is required. Application deletion markers and terminal membership records are retained. All peers hold the same namespace; sharding, transactions, durable event streams and ownership reassignment require additional protocols.

The complete functionality here means the functionality of this declared reference design. Other gossip families include partial-view peer sampling, aggregation, multicast trees and causal or CRDT replication. They can replace modules, but are not silently implied by the baseline.

How the explanations and algorithms fit together

Every numbered section starts with its own description of the functionality and the terms in its heading. Detailed explanations stay with the corresponding algorithm. Sections 8 and 9 cover the request and reply halves of probing; sections 12 and 13 cover both sides of state repair. Read the functional description and example first, then follow the decisions in the pseudocode.

2 Modules and dependencies

A module is a part of the program with one defined responsibility. A dependency is another part it calls to get work done. This division makes each responsibility easier to explain, implement and change.

For example, the probe module asks whether B can respond. It uses the transport module to send its question. If the check fails, it asks the membership module to record suspicion. The diagram and table show which module owns each task and which operations other modules may request.

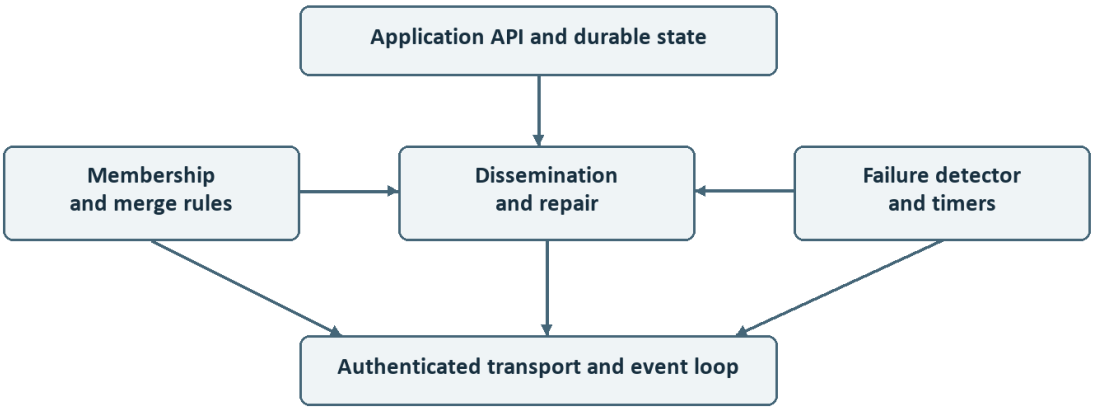


Figure 2 All protocol events enter one serialized state machine. Transport workers and snapshot readers return results as events.

Module	Owns	Public operations
Runtime	Lifecycle and event scheduling	start dispatch stop
Membership	Member records and transitions	mergeMember candidates snapshot
Probe engine	Pending probes and helper requests	probeTick onAck onTimeout
Dissemination	Bounded queue of current rumors	enqueue pack commitSend
Application store	Durable values and logical clock	write mergeValue read snapshot
Repair engine	Reliable bounded sessions	bootstrap repairTick reconcile
Transport	Encoding authentication and resource limits	sendDatagram openStream close
Observability	Metrics and asynchronous notifications	count observe emit

Dependencies flow through interfaces. The probe engine requests membership transitions; it does not edit tables directly. Transport verifies envelopes before dispatch. Application writes and received records use one merge policy. The repair path calls the same merge functions as the fast path.

Concurrency contract

Concurrency means that activities can be in progress during the same period, such as a network read and a storage write. This contract defines which activity may change protocol state and how other activities report their results.

Exactly one event loop owns mutable protocol state. Timer callbacks carry tokens and post events; they never mutate state directly. Disk or stream work may run outside that loop, but completions return to it. A storage transaction commits both a local logical clock and its value before success is reported. Callbacks execute outside the loop and cannot block probes.

3 Types and persistent state

A type describes the fields that a piece of information contains. State is the information a node remembers between actions. Persistent state is information committed to storage so that it can be recovered after the program stops and restarts.

This section defines the records the protocol uses and separates facts that must survive a crash from temporary work held in memory. A stored colour value and its version must survive restart. A request waiting for a reply can be discarded when its process stops. Separating these cases prevents a restarted process from accidentally reusing an old request or losing an acknowledged stored value.

The records in ordinary language

OwnerAnnouncement is the process's own description of itself: its identity, address, incarnation and metadata. Metadata is a bounded collection of descriptive fields, such as a role or software version. MemberRecord adds a status report about that owner announcement. The owner signs its own ALIVE or LEFT statement; another admitted node may report SUSPECT or DEAD.

PrincipalID names an authorized identity, such as a service account. InstanceID names one running lifetime of a process. ValueRecord holds one key, its value or deletion marker, and its version stamp. Envelope packages records together with a message kind. A signature is evidence that the holder of an authorized signing key authored the signed fields.

Volatile durable and immutable state

Volatile state exists only in memory and disappears on a crash. Pending probes and rumor queues can be rebuilt or abandoned. Durable state survives a process crash after the storage system confirms a commit. Values, the logical counter and retained membership records need that protection. An immutable snapshot is a frozen read view used while live state continues to change.

Why one event loop owns mutations

An event loop takes one queued event at a time and runs its handler. Network workers and timers place events in this queue. This keeps two handlers from simultaneously changing membership. Slow disk or network work runs separately and reports its result back. A callback is a notification to application code; it must not block protocol progress.

Follow one timer token

A token identifies one scheduled action. Suppose B is suspected at incarnation 4 and a timer receives token T1. B later refutes at 5. Canceling T1 may not remove an expiry event already waiting in the queue. Checking that T1 is still the currently registered token makes the stale event harmless.

Pseudocode

```
type InstanceID = random 128-bit identifier per process start
type PrincipalID = authenticated service identity
enum Status { ALIVE, SUSPECT, DEAD, LEFT }
rank = { ALIVE: 0, SUSPECT: 1, DEAD: 2, LEFT: 3 }

type OwnerAnnouncement {
  cluster, instance, principal, endpoint, incarnation,
  metadata, ownerSignature
}
type MemberRecord {
  owner: OwnerAnnouncement,
  status, reporter, reportSignature
}
# ALIVE and LEFT must be authored by the owner.
# SUSPECT and DEAD bind a reporter to an owner announcement.
```

```
type Stamp { counter: unsigned integer, writer: InstanceID }
type ValueRecord {
  cluster, namespace, key, stamp, deleted, value,
  writerPrincipal, writerSignature
}
type Envelope {
  cluster, protocolVersion, senderInstance, senderPrincipal,
  messageID, kind, body, rumors, authentication
}

state Runtime {
  mode: STARTING | JOINING | RUNNING | DRAINING | STOPPED,
  selfOwner, members, suspicionTimers, probeBag,
  pendingProbes, pendingRelays, rumorQueue, replayWindow,
  activeSessions, timerTokens, randomGenerator
}
durable AppStore { valuesByKey, logicalCounter }
durable MembershipStore { current records including terminal ones }
```

A canonical encoding fixes field order, integer representation and signature input. Signatures cover the cluster and all record fields. Relayed records retain their original signatures; an authenticated envelope alone does not prove who authored a relayed record.

An owner announcement authorizes one instance and incarnation. A reporter cannot manufacture a larger incarnation for a target. The admission policy binds principals to allowed endpoints and namespaces and limits active instances. Only the owner advances its incarnation, before publishing the corresponding announcement.

Notation: r.instance and r.incarnation abbreviate r.owner.instance and r.owner.incarnation for member records; self refers to selfOwner. A candidate carries its signed owner endpoint. Comparison of stamps always means stampOrder. Boolean validation helpers return false on invalid input; require raises a handled rejection. No invalid record can mutate state.

Pending probes, replay windows, rumor queues and sessions are bounded volatile state. Values and retained member records survive a crash. A new process signs a fresh instance announcement and bootstraps before exposing reads.

4 Configuration and platform contracts

Configuration is the set of adjustable values that controls how the program runs, such as how often it sends a probe and how long it waits for a reply. The platform is the programming environment supplying clocks, network communication, storage and task scheduling. A contract states what one of those supplied operations must do.

For example, the protocol asks a timer to report that a deadline has passed; the platform chooses how to implement the timer. It asks storage to commit several changes together; the storage implementation must either commit all of them or fail without a partial commit. The settings below describe the protocol's timing and resource budgets, while the operation contracts describe what an implementation must supply.

Setting	Illustrative starting value	Meaning
probePeriod	1 second	Time between local probe starts
directTimeout	250 milliseconds	Wait for a direct acknowledgment
indirectTimeout	500 milliseconds	Additional wait after helper requests
indirectFanout	3 peers	Helpers per probe at most
gossipPeriod and fanout	200 milliseconds and 3	Independent fast dissemination
suspicionTimeout	5 x probePeriod x max(1 ln(N+1))	Local suspicion grace interval
rumorSendLimit	4 x ceil(log2(max(2 N)))	Send attempts per current record

Setting	Illustrative starting value	Meaning
repairPeriod	30 seconds with jitter	Periodic state reconciliation
maxDatagramBytes	1200 bytes total	Including framing and authentication
maxStreamPageBytes	64 KiB	Bounds each streamed page
maxConcurrentRepairs	2	Bounds snapshot and stream resources

These are illustrative reference settings, not universal defaults or production recommendations. N counts admitted ALIVE or SUSPECT instances. Choose deadlines from measured network latency and local scheduling delay; ensure directTimeout plus indirectTimeout is less than probePeriod. Independent gossip can run between probes.

Required platform operations

Platform operations are the services supplied by the chosen programming environment. The protocol calls them to read time, schedule future work, store records and communicate; their implementations must meet the meanings listed below.

Pseudocode

```
Clock.now()                  # monotonic local time
Timers.schedule(token, at)    # posts expiry(token)
Timers.cancel(token)          # expiry may already be queued
Random.shuffle(items)        # unbiased independent local randomness
Storage.transaction(action)  # durable atomic commit or failure
Snapshot.open(store)          # immutable indexed view until closed
Transport.send(packet)       # bounded attempt; never blocks event loop
Transport.stream(peer)       # authenticated ordered duplex session
Auth.validate(record)        # signature plus admission and size policy
Metrics.emit(event)           # asynchronous and bounded
```

scheduleJittered installs recurring events with bounded jitter; cancelPeriodic stops them. recoveryCadence is 10: every tenth repair selection cycles through seeds or retained non-LEFT peers. Its counter starts at zero. Token timers are one-shot; canceled expiry events may already be queued. Cap metadata so complete signed control messages fit the datagram limit.

Every queue, map of pending requests, record size and snapshot lifetime has an explicit deployment limit. Admission and writes fail visibly when retained state cannot fit. Keeping an unbounded history in finite storage is impossible; safe garbage collection requires extra coordination described in section 14.

5 Transport and event dispatch

Transport is the part of the program that sends messages to other nodes and receives messages from them. It turns outgoing information into network bytes and turns incoming bytes back into a message. It checks who sent the message and whether its format and size are acceptable before the message can affect the protocol.

An event is something the program must respond to, such as receiving a message or reaching a timer deadline. Event dispatch means selecting the function that handles that event. A handler is the function containing the response steps. For example, when B receives a PING message from A asking for a reply, dispatch selects onPing; when A receives the reply, dispatch selects onAck.

This section implements the network-message path: receive bytes, validate the message, place it in the event queue, apply any attached updates, then call the appropriate handler. Timer events reach their timer handlers through the scheduler. The transport delivers information; the selected handler decides what that information means for node state.

Inputs and important terms

raw is the received byte sequence. decodeBounded converts it to fields while enforcing limits on lengths and nesting. The protocolVersion identifies a mutually understood message format. The cluster identifier prevents accidentally mixing two clusters. An authenticated channel establishes the peer identity and protects the traffic.

Replay means receiving a previously sent message again. A bounded replay window remembers a recent set of message identifiers. Deduplication avoids repeated work, but the merge rules must also be idempotent: applying the same record twice has the same effect as applying it once. This remains necessary after old replay entries expire.

Follow onNetworkPacket and dispatch

1. Reject an oversized packet before expensive parsing. Decode the remaining packet and verify its sender, cluster, version and rate budget.

2. Drop a recently seen message identifier. Otherwise remember it and queue the message for the event loop.

3. Merge valid attached rumors. Then inspect kind: PING goes to the probe request handler, ACK to its reply handler, and NOTICE to the membership merge. A GOSSIP message exists solely to carry rumors.

Follow sendDatagram

Build the required message body first. Measure its complete authenticated size. Fill only the remaining byte space with rumors, then submit one send attempt. commitSend increases retry counts only when the local transport accepts that attempt. It does not mean a peer received or committed anything.

Small example

A sends B a PING that also carries colour at stamp (8,A). B validates the packet, merges the colour record, then handles the PING and replies. Even if colour was already known, B still processes the probe request. If local congestion prevents A from submitting the packet, its rumor retry count is not spent.

What the limits protect

A rate limit bounds work per sender; a byte budget bounds packet size. Backpressure means refusing or postponing new work when capacity is full. Reserved control capacity keeps probes and ACKs from waiting behind large data transfers. An invalid packet changes no protocol state.

Pseudocode

```
onNetworkPacket(raw, authenticatedChannel):
  if size(raw) > channelLimit: reject OVERSIZE
  envelope = decodeBounded(raw) or reject MALFORMED
  require verifyEnvelope(envelope, authenticatedChannel)
  require envelope.cluster == configuredCluster
  require compatible(envelope.protocolVersion)
  require admitted(envelope.senderPrincipal)
  require ingressRateLimitAllows(envelope.senderPrincipal)
  if replayWindow.contains(envelope.senderInstance,
                           envelope.messageID): return
  replayWindow.rememberBounded(envelope)
  postToEventLoop(envelope)
```

```
dispatch(envelope):
  if mode == STOPPED: return
  for record in envelope.rumors:
    if validMember(record): mergeMember(record)
    else if validValue(record): mergeValue(record)
    else: Metrics.count(INVALID_RECORD)
  switch envelope.kind:
    PING:       onPing(envelope)
    ACK:        onAck(envelope)
    PING_REQ:   onPingRequest(envelope)
    RELAY_ACK:  onRelayAck(envelope)
    GOSSIP:     pass              # rumors were already merged
    NOTICE:     mergeMember(envelope.body) if validMember(envelope.body)
    otherwise: reject UNSUPPORTED_KIND

sendDatagram(peer, kind, body, priority = NORMAL):
  base = newEnvelope(kind, body, freshMessageID())
  require authenticatedEncodedSize(base) <= maxDatagramBytes
  picked = packRumors(remainingAuthenticatedBytes(base))
  packet = authenticate(encode(base with rumors = picked))
  if Transport.trySend(peer, packet, priority):
    commitSend(picked)             # network attempt, not delivery
  else: Metrics.count(LOCAL_SEND_REJECTED)
```

Reserve transport and event queue capacity for probes and acknowledgments. Data repair must not starve them. Large values travel on reliable streams; only records that fit a datagram enter its rumor queue. Queue eviction does not delete retained state.

Replay suppression limits repeated transport work, but correctness comes from idempotent merges. Once a replay-window entry expires, processing the old packet again must remain harmless. Authenticated streams apply the same validation to every frame and record.

6 Startup and cluster admission

Startup is the procedure that prepares a new process to participate. A cluster is the group of participating processes. Admission is the decision that an identified process is permitted to join that group. Bootstrap is the initial exchange through which the newcomer obtains existing membership and application information.

A seed is a configured address the newcomer can try first. For example, C contacts seed B, proves its permitted identity, and exchanges stored state with B. After that succeeds, C enables its local application operations and begins regular probes, gossip and repair. An unreachable seed causes another attempt; it does not silently authorize C to create a separate cluster.

Seeds admission and bootstrap

A seed is a configured starting address. It helps a node discover the cluster; it is not automatically a leader or a special authority. Admission is the decision that this authenticated process is allowed to join, within identity and capacity policies. Bootstrap is the initial transfer and merge of cluster state.

The lifecycle variable mode prevents operations at the wrong time. STARTING initializes storage and listeners. JOINING attempts admission and state exchange. RUNNING enables application operations and regular background work. DRAINING rejects new local work while leaving. STOPPED has released its resources.

Follow the startup functions

1. start loads retained stores, creates a fresh process identifier, signs its first owner announcement and opens listeners. A listener is the local endpoint waiting for incoming connections or packets.

2. onBootstrapSession sends JOIN to a seed. A nonce is a fresh challenge value used to correlate the reply with this request. A rejection or failed connection leads to another attempt.

3. The seed validates the requested identity and capacity in onJoinRequest, returns JOIN_ACCEPT, and participates in the repair protocol over the same session.

4. After successful reconciliation, enterRunning schedules probes, gossip and repair and enables the local API.

How retry behaves

Backoff increases the delay between failed attempts up to a cap. Jitter adds a small random variation, preventing many nodes from retrying at the same instant. The exception is an explicitly authorized new cluster: its first node can enter RUNNING without discovering an existing peer.

Small example and result

C tries seed A, which is offline, then seed B, which accepts it. C learns B's membership and colour records and begins regular participation. B may receive a newer colour write during this exchange; C can learn that write later. Bootstrap makes C ready to participate, not a globally up-to-date read barrier.

Pseudocode

```
start(seedEndpoints, bootstrapNewCluster = false):
  require mode == STARTING
  loadDurableStoresAndValidate()
  instance = randomInstanceID()
  selfOwner = signOwner(instance, incarnation = 0, endpoint)
  openAuthenticatedListeners()
  mode = JOINING
  if bootstrapNewCluster:
    require explicitlyAuthorizedNewCluster()
    mergeMember(ownerRecord(selfOwner, ALIVE))
    enterRunning()
    return
  trySeedsWithBackoff(seedEndpoints)

onBootstrapSession(seed, bootstrapStream):
  require authenticatedAndAuthorized(seed)
  send JOIN(selfOwner, cluster, protocolVersion, nonce)
  response = receive JOIN_ACCEPT or JOIN_REJECT by deadline
  if rejected: closeSession(); tryNextSeed(); return
  require acquireSessionSlot() or closeAndRetrySeed()
  # Reconcile both member and application snapshots in section 12.
  if not reconcile(seed, bootstrap = true, session = bootstrapStream):
    closeSession(); tryNextSeed(); return
  mergeMember(ownerRecord(selfOwner, ALIVE))
  enterRunning()

onJoinRequest(session, request):
  require mode == RUNNING
  require validateOwner(request.selfOwner)
  require admissionCapacityAllows(request.selfOwner)
  require compatible(request.protocolVersion)
  mergeMember(ownerRecord(request.selfOwner, ALIVE))
  send JOIN_ACCEPT(request.nonce, negotiatedLimits)
  serveReconcile(session)

enterRunning():
  mode = RUNNING
  scheduleJittered(PROBE_TICK, probePeriod)
  scheduleJittered(GOSSIP_TICK, gossipPeriod)
  scheduleJittered(REPAIR_TICK, repairPeriod)
  enableLocalApplicationAPI()
```

Seeds provide initial contact, not permanent leaders. After admission, any reachable authorized member may serve bootstrap. An unreachable seed list causes retry with capped exponential backoff and jitter; it never silently creates an independent cluster.

Bootstrap is a reconciled snapshot, not a globally current read barrier. Concurrent writes may arrive later. A partially admitted instance whose join fails may be learned by others and later suspected; retries reuse the same process instance until that process stops.

7 Membership merge and refutation

Membership is a node's locally stored knowledge of which process instances belong to the group and what status they have. Merge means comparing an incoming report with the local report and deciding which one to keep. Refutation means that a running process answers a suspicion about itself by publishing a newer statement that it is alive.

An incarnation number is a version for one process's announcements during its lifetime. For example, A may store B as SUSPECT at incarnation 4, meaning it doubts that B is available. B can refute that report by announcing ALIVE at incarnation 5. A then keeps the newer announcement. This section defines that comparison, saves the winning report, spreads it to peers, and starts or cancels the waiting period associated with suspicion.

Owner and reporter are different roles

The owner is the process being described. The reporter authors a status statement. B authors its own ALIVE or LEFT report. A can author a SUSPECT report about B, but it must attach B's genuine owner announcement. A cannot invent a new incarnation for B.

How the precedence rule works

Within one incarnation, the reference order is ALIVE below SUSPECT below DEAD. A higher owner-signed incarnation outranks a lower one. LEFT has a special rule: it is terminal for that process instance. Terminal means no later message revives that same instance; a new process lifetime needs a new identifier.

This is a merge policy, not a measurement of objective health. The rank table makes comparisons deterministic. Reports with the same version do not create a fresh suspicion deadline. Conflicting owner announcements at the same incarnation are rejected rather than resolved arbitrarily.

Follow mergeMember

1. Validate signatures and admission, then look up the locally retained record for that instance.

2. Check whether the incoming report is about this node itself. If it is a current SUSPECT or DEAD report, publish an ALIVE announcement with a larger incarnation. That is refutation.

3. For another instance, ignore an older or equal report. Persist a winning report before making it the in-memory current record.

4. Cancel any obsolete suspicion timer. Queue the new report for spreading. If the winning status is SUSPECT, start a new token-bound timer and notify the target promptly.

Small example

A stores B as ALIVE at 4. A then learns SUSPECT at 4 and keeps it because suspicion wins at equal incarnation. A delayed ALIVE at 4 loses. B hears the suspicion and signs ALIVE at 5; that wins over every nonterminal report at 4. A queued timer for 4 cannot kill B at 5 because its token and incarnation no longer match.

What an ACK cannot do

A successful probe can finish a pending request, but it does not erase SUSPECT at the same incarnation. The ACK must carry a genuinely newer owner announcement to change that membership result.

Pseudocode

```
memberOrder(r) = (r.owner.incarnation, rank[r.status])
```

```
mergeMember(incoming):
  require validMember(incoming)   # signatures and admission
  id = incoming.owner.instance
  old = members.get(id)
  if old != NONE and old.incarnation == incoming.incarnation:
    require canonicalOwner(old) == canonicalOwner(incoming)
  if old != NONE and old.status == LEFT: return
  if id == selfOwner.instance:
    if mode == DRAINING or mode == STOPPED: return
    if incoming.status in {SUSPECT, DEAD} and
       incoming.owner.incarnation >= selfOwner.incarnation:
      selfOwner = signOwner(id,
        incoming.owner.incarnation + 1, currentEndpointAndMetadata)
      mergeOwnedAlive(selfOwner)
      return
  if old != NONE and incoming.status != LEFT and
     memberOrder(incoming) <= memberOrder(old): return
  # LEFT is terminal for this instance, even against later reports.
  persistMemberBeforeApplying(incoming) or enterStorageFailure()
  members[id] = incoming
  cancelSuspicionTimer(id)
  invalidateProbeBagIfEligibilityChanged(old, incoming)
  enqueue(incoming)
  if incoming.status == SUSPECT:
    token = freshToken(id, incoming.owner.incarnation)
    suspicionTimers[id] = token
    schedule(token, now() + suspicionTimeout(activeCount()))
    sendDatagram(incoming.owner.endpoint, NOTICE,
                 incoming, priority = CONTROL)
  emitMembershipChangeAsync(old, incoming)

mergeOwnedAlive(owner):
  alive = ownerRecord(owner, ALIVE)
  persistMemberBeforeApplying(alive) or enterStorageFailure()
  members[owner.instance] = alive
  cancelSuspicionTimer(owner.instance)
  enqueue(alive)
```

Within an incarnation, DEAD dominates SUSPECT and SUSPECT dominates ALIVE. A higher owner-signed incarnation supersedes earlier nonterminal reports. This reference design permits an active owner to refute DEAD as well as SUSPECT. LEFT is an owner-signed terminal statement; a departing instance cannot resurrect itself. These are explicit reference semantics, not a universal SWIM wire rule.

Equal-version duplicate reports do not restart suspicion timers. A plain ACK proves a probe succeeded but never resets membership to ALIVE at the same incarnation. Refutation requires a newer owner announcement. Reject conflicting owner announcements at the same incarnation as a protocol violation.

8 Direct and indirect probes

A probe is a request sent to another node to check whether it can reply. A direct probe goes from the checking node to the target itself. An indirect probe asks other nodes, called helpers, to try contacting the same target through their own network paths.

For example, A first asks B for a reply. If none arrives before A's deadline, A asks C to try B. If C obtains B's reply and relays it in time, A completes the check successfully. If both stages fail, A records suspicion about the version of B it checked. This section creates those checks and handles their deadlines; the next section processes the replies.

The four message kinds

PING asks its receiver to reply. ACK is the target's signed reply. PING_REQ asks a helper to probe a named target. RELAY_ACK carries the target's reply back through that helper. The helper is an alternate path, not a substitute for a response from the target.

A pending probe is a temporary record of a request still awaiting an answer. It includes a unique token, the target, its observed incarnation and the current phase. DIRECT and INDIRECT identify which wait is underway. A helper keeps its own pending relay, linking its fresh probe token to the original requester and token.

Follow probeTick and the timeouts

1. Choose a peer from a shuffled bag of eligible instances. Taking peers out of this bag before reshuffling gives every stable member a turn.

2. Save the pending probe, send PING and start the direct timeout. An ACK must name the correct token, requester and target.

3. If that deadline passes, retain the probe and ask a bounded number of distinct helpers. Start the additional indirect timeout.

4. If no valid reply arrives by then, remove the pending probe. Suspect only the same incarnation that was originally probed, because a newer owner announcement makes the old attempt obsolete.

Follow the reply handlers

onPing creates a target proof and replies. onPingRequest creates a separate helper token and probes the target. onAck can complete a local probe or forward a helper result. onRelayAck checks the selected helper and target proof before completing the original probe. Completion removes pending state and cancels its timer.

Follow the suspicion timer

If a SUSPECT report survives until its deadline, onSuspicionExpiry checks both incarnation and token before publishing DEAD. A helper timeout only frees relay state; it does not itself declare the target dead.

Small example

A's direct reply from B is lost. A asks C, C receives B's reply, and C relays it to A before the indirect deadline. A completes the probe without creating a new suspicion. If A already knew an equal-incarnation suspicion from elsewhere, this reply alone does not cancel it; B must refute with a newer incarnation.

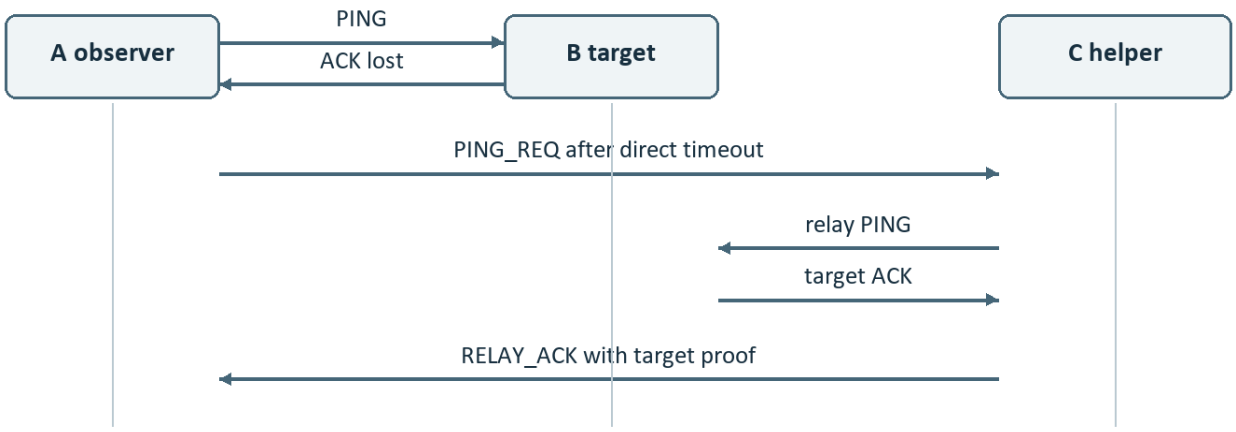


Figure 3 A helper provides an alternate path to the target. A helper acknowledgment alone is insufficient; the target must produce the acknowledgment proof.

Pseudocode

```
probeTick():
  if mode != RUNNING: return
  if pendingProbes.size >= maxPendingProbes: countSkip(); return
  target = nextFromShuffledBag(ALIVE or SUSPECT, excludingSelf)
  if target == NONE: return
  p = Probe(freshToken(), target.instance, target.incarnation,
            phase = DIRECT, helpers = emptySet)
  pendingProbes[p.token] = p
  sendDatagram(target, PING, (p.token, self.instance), CONTROL)
  schedule(p.token, now() + directTimeout)

onProbeTimeout(token):
  p = pendingProbes.get(token)
  if p == NONE: return
  if p.phase == DIRECT:
    p.phase = INDIRECT
    p.helpers = randomDistinctCandidates(indirectFanout,
                excluding = {self.instance, p.target})
    for helper in p.helpers:
      sendDatagram(helper, PING_REQ, (p.token, p.target), CONTROL)
    schedule(token, now() + indirectTimeout)
  else:
    pendingProbes.remove(token)
    current = members.get(p.target)
    if current != NONE and current.status in {ALIVE, SUSPECT}
       and current.incarnation == p.observedIncarnation:
      mergeMember(reporterRecord(current.owner, SUSPECT))
```

The shuffled bag visits known eligible peers without replacement, then reshuffles. Newly eligible members invalidate the bag. Under stable membership this avoids indefinite random starvation. There is no unconditional real-time failure bound under unbounded pauses or network delay.

9 Probe replies and suspicion timers

A probe reply is the target's answer to a request checking whether it can respond. The protocol calls that reply an ACK, meaning acknowledgment. A suspicion timer is a scheduled waiting period during which a suspected process can publish newer evidence that it is alive before the local node marks it DEAD.

This section handles incoming questions, direct replies, helper requests, and replies forwarded through helpers. It also handles the end of the suspicion period. A token is an identifier for one particular request or scheduled action. For example, a reply for probe P1 can complete P1 but cannot complete a later probe P2; a timer for incarnation 4 cannot mark incarnation 5 dead. The handlers verify these identities before changing state.

Pseudocode

```
onPing(e):
  if mode not in {RUNNING, JOINING}: return
  proof = signTargetAck(e.body.token, e.senderInstance,
                        selfOwner.instance, selfOwner)
  sendDatagram(e.sender, ACK, proof, CONTROL)
```

```
onAck(e):
  require validTargetAck(e.body)
  mergeMember(ownerRecord(e.body.owner, ALIVE))
  p = pendingProbes.get(e.body.token)
  if p != NONE and e.body.target == p.target and
     e.body.requester == self.instance:
    completeProbe(p.token)
  r = pendingRelays.get(e.body.token)
  if r != NONE and e.body.target == r.target and
     e.body.requester == self.instance:
    sendDatagram(r.requester, RELAY_ACK,
                 (r.originalToken, e.body), CONTROL)
    releaseRelay(r.token)

onPingRequest(e):
  require admittedTarget(e.body.target) and relayBudgetAllows()
  r = Relay(freshToken(), e.sender, e.body.token, e.body.target)
  pendingRelays[r.token] = r
  sendDatagram(r.target, PING, (r.token, self.instance), CONTROL)
  schedule(r.token, now() + helperTimeout)

onRelayAck(e):
  p = pendingProbes.get(e.body.originalToken)
  if p == NONE or e.senderInstance not in p.helpers: return
  require validTargetAck(e.body.targetProof)
  require e.body.targetProof.target == p.target
  require e.body.targetProof.requester == e.senderInstance
  mergeMember(ownerRecord(e.body.targetProof.owner, ALIVE))
  completeProbe(p.token)

onSuspicionExpiry(token):
  r = members.get(token.instance)
  if suspicionTimers.get(token.instance) != token: return
  if r.status != SUSPECT or r.incarnation != token.incarnation: return
  mergeMember(reporterRecord(r.owner, DEAD))

completeProbe(token): cancelTimer(token); pendingProbes.remove(token)
releaseRelay(token): cancelTimer(token); pendingRelays.remove(token)
```

Helper timeout only releases relay state; it does not independently suspect the target. ACK correlation binds the token, target and requester. Timer tokens prevent canceled or obsolete expiry events from killing a refuted instance. An indirect acknowledgment must arrive before its pending probe is released to count as success.

10 Dissemination and bounded rumors

Dissemination means spreading information to other participants. A rumor is a protocol record waiting to be spread, such as a changed value or a membership report. Bounded means that the program limits the memory, packet space and send attempts it uses for this work.

This section maintains a queue of pending updates, chooses which records fit in each outgoing message, and stops retrying a record after its send budget is spent. For example, A can queue colour version 8, include it in several messages, and replace that pending entry when colour version 9 arrives. Removing the queue entry stops the fast send attempts; the retained value remains available for later state repair.

Queue entries and superseding records

A queue is a collection of work waiting to be done. A rumor entry holds a record, how often it has been sent, and its send limit. recordKey groups records about the same thing: one process instance or one application key. To supersede means to make an earlier record unnecessary under the declared merge rules.

For example, a newer colour value replaces an older colour rumor. A duplicate does not reset its send count. Removing a rumor entry means stop trying the fast path; it does not remove the corresponding stored application value or membership record.

Follow the queue operations

1. enqueue rejects a record from the datagram queue if it is too large. Its retained copy can still travel through stream repair.

2. If the record wins over the currently queued record for the same key, make room within queue limits and insert a fresh entry. Eviction removes pending fast-path work to free capacity.

3. packRumors selects records that fit the byte budget. Fair lanes reserve service for membership and application records; otherwise a busy membership stream could starve values.

4. After the transport accepts a packet, commitSend advances the counts of the exact records included. At the limit, remove their queue entries.

What gossipTick contributes

gossipTick sends separate GOSSIP packets to a small random peer sample even when no probes are occurring. Probe and ACK packets can also piggyback the same queue's records. Each actual packet has its own message identifier, so replay suppression does not confuse contacts with different peers.

Small example and result

A queues colour version 8 with a limit of four attempts. Two accepted packets carry it. Before the next packet, version 9 wins locally and replaces that queued entry. A now spends future attempts on 9. If C misses all of them, repair can still fetch retained version 9. A send limit bounds effort; it never certifies delivery to every node.

Pseudocode

```
recordKey(r):
  if isMember(r): return (MEMBER, r.owner.instance)
  return (VALUE, r.namespace, r.key)

enqueue(r):
  key = recordKey(r)
  if encodedSize(r) > maxRumorRecordBytes:
    markRepairNeeded(); return
  old = rumorQueue.get(key)
  if old != NONE and not supersedes(r, old.record): return
  while queueWouldExceedLimits(key, r):
    evictFairlyFromRumorQueue(); markRepairNeeded()
  rumorQueue[key] = Entry(r, sent = 0,
    limit = rumorSendLimit(activeCount()), age = freshSequence())

packRumors(byteBudget):
  result = []
  # Fair lanes for membership and values; rotate within each lane.
  for entry in fairOrderByFewestSendsThenAge(rumorQueue):
    if encodedSize(result + entry.record) <= byteBudget:
      result.append(entry.record)
  return result

commitSend(records):
  for r in records:
    entry = rumorQueue.get(recordKey(r))
    if entry == NONE or entry.record != r: continue
    entry.sent += 1
    if entry.sent >= entry.limit: rumorQueue.remove(recordKey(r))

gossipTick():
  if mode != RUNNING: return
  peers = randomDistinctCandidates(gossipFanout, excludingSelf)
  for peer in peers: sendDatagram(peer, GOSSIP, emptyBody)
  # Independent per-peer packets get fresh message identifiers.
```

supersedes uses the membership ordering in section 7 or the value ordering in section 11. LEFT supersedes every nonterminal record for its instance. Identical records do not reset retransmission counts. Replacing a queued rumor discards only superseded information.

A send limit bounds effort, not delivery probability. Zero reachable peers leaves rumors pending; exhausted or evicted rumors remain recoverable from retained snapshots. packRumors accounts for encoding, signatures, framing and authentication overhead. Probe bodies and urgent notices must fit even when the rumor budget is zero.

This queue is inspired by transmit-limited broadcast queues [4], but the fair-lane scheduler is a reference-design choice. A busy node must reserve some dissemination bandwidth for values as well as membership, otherwise values can starve until repair.

11 Application state and conflict policy

Application state is the data the service stores for its users or its own work, separate from membership information. This reference uses named keys with associated values, such as colour equals blue. A conflict occurs when different nodes produce competing updates to the same key. The conflict policy is the rule used to choose the stored result.

The rule here assigns each write a stamp containing a logical counter and a writer identifier. A logical counter is an integer used to order observed updates; it is not a clock measuring seconds. Every node compares stamps in the same way and retains the largest. For example, replicas that receive red at stamp (12,A) and blue at (13,B) both keep blue regardless of delivery order. Deletion is also a stamped record, so an offline replica can learn that a value was removed.

What a logical stamp measures

The counter is stored durably. A local write increments it and pairs it with this process's writer identifier. Receiving a valid record raises the local counter to at least the received counter. Consequently a later local write gets a larger counter than a record already observed. The number does not reveal elapsed time or which disconnected wall-clock write happened last.

If both counters are equal, compare the writer identifiers as unsigned byte sequences. This lexicographic order is a fixed tie breaker, like comparing words character by character. It gives every node the same answer but does not mean the winning writer is more important.

Follow write read and mergeValue

1. write checks permission and capacity. A durable transaction stores the advanced counter and signed value together. Only after commit does it report LOCAL_COMMITTED and queue the rumor.

2. mergeValue verifies a received record and compares its stamp with the retained record. Older stamps lose. Identical records do nothing. An equal stamp with different contents is a violation, not an ordinary conflict.

3. A winning received record is stored durably, queued for further spreading, and announced through a local callback. read returns the locally retained winner or ABSENT for a missing key or tombstone.

Why repeated delivery is safe

Idempotent means duplicates have no additional effect. Commutative means delivery order does not change the winner. Associative means grouping several merges differently does not change the winner. Choosing the maximum valid stamp has these properties. They make retry and repair safe for stored state; they do not make external callback effects execute exactly once.

Small example including a deletion

A writes red at (12,A) while B writes a tombstone at (12,B). If B sorts after A, every replica eventually keeps the tombstone. A later observes it and writes blue at (13,A), which wins. The losing red value is discarded by this policy. Applications that need to preserve both concurrent edits must replace this merge module with an appropriate conflict representation.

Pseudocode

```
stampOrder(s) = (s.counter, unsignedLexicographic(s.writer))

write(key, value, deleted = false):
  require mode == RUNNING and authorizedLocalWrite(key)
  require recordSizeAllowed(key, value)
  transaction:
    require storageCapacityAllows(key, value)
    counter = checkedIncrement(AppStore.logicalCounter)
    r = signValue(key, Stamp(counter, self.instance),
                  deleted, NONE if deleted else value)
    AppStore.logicalCounter = counter
    AppStore.valuesByKey[key] = r
  enqueue(r)                        # best effort after durable commit
  emitValueChangeAsync(r)
  return LOCAL_COMMITTED(r.stamp)   # no remote delivery assertion

mergeValue(r):
  require validValue(r) and authorizedWriter(r)
  transaction:
    AppStore.logicalCounter = max(AppStore.logicalCounter,
                                  r.stamp.counter)
    old = AppStore.valuesByKey.get(r.key)
    if old != NONE and stampOrder(r.stamp) < stampOrder(old.stamp):
      return UNCHANGED
    if old != NONE and r.stamp == old.stamp:
      require canonicalBytes(r) == canonicalBytes(old)
      return UNCHANGED
    require storageCapacityAllows(r.key, r.value)
    AppStore.valuesByKey[r.key] = r
  enqueue(r)
  emitValueChangeAsync(r)
  return CHANGED

read(key):
  require mode == RUNNING
  r = AppStore.valuesByKey.get(key)
  if r == NONE or r.deleted: return ABSENT
  return (r.value, r.stamp)          # locally known state
delete(key): return write(key, NONE, deleted = true)
```

Each key is a maximum-by-stamp register. The merge is associative, commutative and idempotent for valid nonconflicting records. A write made after observing a record gets a larger counter. Concurrent writes are ordered by the instance identifier; the losing value is intentionally discarded. This is deterministic overwrite semantics, not business-level conflict resolution or physical last-write time.

Replacing this module with a CRDT or multivalue register requires changing the inventory and supersedes contracts too. The reference register gives no multi-key atomicity, compare-and-set, exactly once callbacks or durable event history. Counter overflow fails explicitly; it never wraps.

12 Bidirectional anti entropy

Anti entropy is the periodic comparison of stored information to find and repair differences between nodes. Bidirectional means both nodes can obtain missing information from the other. This is how the protocol recovers updates that the limited fast gossip attempts did not deliver.

Each side compares an inventory, which is a list of its stored keys and versions. The comparison uses a snapshot, a stable view of its data while the exchange runs. For example, if A holds colour at version 8 and B holds it at 5, B requests version 8. If B has another key that A lacks, A requests that key in the reverse direction. Received records pass through the same comparison and storage rules used for ordinary gossip.

The comparison vocabulary

An inventory lists each key, its version stamp and a hash of its canonical record bytes. A hash is a compact fingerprint used to check that the transferred bytes match the advertised record. It does not establish authorization; signatures and admission checks still do that. Canonical means both nodes encode the same logical record in the same fixed way.

A snapshot is a stable read view taken at one point in the local store's history. Pinning keeps that view available until the session finishes. A snapshot token identifies that particular view, so a request for version 8 does not accidentally fetch live version 9 after another write. A session is one bounded conversation over an authenticated stream.

Follow reconcile

1. Reserve capacity, connect, agree on compatible limits, and open immutable local snapshots. Exchange the tokens naming both sides' snapshots.

2. Exchange membership records in pages and feed them through the ordinary membership merge. A page is a size-limited portion of a larger inventory or record collection.

3. Read the other value inventory. If a key is absent locally or the other stamp is newer, request that exact pinned record, verify it, and wait for its merge to commit.

4. Both sides perform the same comparison. Exchange completion markers, then release the snapshots, stream and session slot even if an error occurs.

What serveReconcile does

The serving side answers GET_RECORD from its pinned snapshot while also driving its own fetch direction. Large records use bounded chunks, or successive pieces. Multiplexing lets requests and replies for both directions share the stream without one side blocking the other. Cleanup in finally runs on success, timeout or failure.

Small example and result

A holds colour at 8 and B holds it at 5. B sees 8 in A's inventory and fetches it. If B also holds a key absent from A, A fetches that key in the reverse direction. A new write at 9 after the snapshots opened can wait for later gossip or repair. A completed session reconciles those snapshots; it does not freeze or synchronize the whole cluster.

Recovering a partition

Occasional seed or retained-peer contact reaches across a healed network split even if peers had marked each other DEAD. A Merkle tree is an optional hierarchy of hashes that helps find differing subsets without scanning the whole inventory. The baseline uses a full scan for clarity.

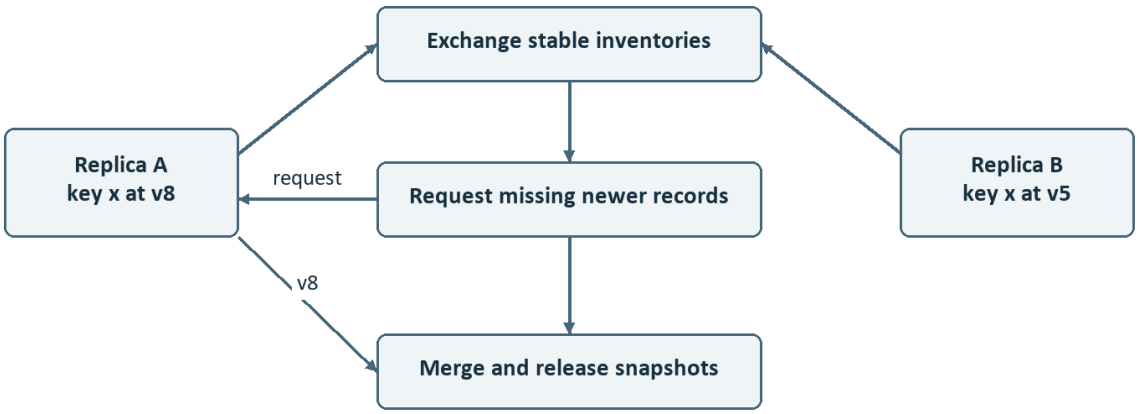


Figure 4 Stable inventories identify differences; only the missing or newer records are fetched. The same merge functions repair either direction.

Pseudocode

```
repairTick():
  if mode != RUNNING or activeSessions >= maxConcurrentRepairs: return
  peer = chooseRepairPeer()
  if peer != NONE:
    acquireSessionSlot()          # reserve on event loop before worker
    startWorker(reconcile(peer, bootstrap = false))

reconcile(peer, bootstrap, session = NONE):
  try:
    if session == NONE:
      session = openAuthorizedStream(peer, sessionDeadline)
    negotiate(cluster, protocolVersion, namespace, byteLimits)
    local = openImmutableSnapshots(members, AppStore)
    remote = exchangeSnapshotTokens(session, local.tokens)
    exchangeMemberPagesAndMerge(session, local.members, remote.members)
    # Both endpoints execute these next steps against pinned snapshots.
    for page in remote.valueInventoryPages(maxStreamPageBytes):
      require validSortedInventoryPage(page, remote.valueToken)
      for item in page:
        ours = local.values.lookup(item.key)
        if ours == NONE or item.stamp > ours.stamp:
          request = (remote.valueToken, item.key, item.stamp, item.hash)
          r = fetchPinnedRecord(session, request)
          require matchesInventory(r, item) and validValue(r)
          postMergeAndAwaitCommit(r)
        else if item.stamp == ours.stamp:
          require item.hash == canonicalHash(ours)
    exchangeCompletionMarkers(session)
    return SUCCESS
  catch timeout, protocolError, storageFailure:
    Metrics.count(REPAIR_FAILED)
    return FAILURE
  finally:
    releaseAllSnapshots(); closeStream(); releaseSessionSlot()
```

An inventory contains every retained key, its stamp and a canonical record hash, including deletions. Each endpoint fetches records where the other snapshot is newer; therefore records known only to either side also cross. Inventory comparison uses stampOrder, not arrival time. Membership exchange sends every retained record in bounded pages and invokes mergeMember on the event loop.

This readable baseline scans the complete inventory. It is not a Merkle tree algorithm. Dynamo uses Merkle trees to narrow repair to differing subtrees [5]; that can replace the inventory module when full scans become expensive.

13 Repair sessions and partition recovery

A repair session is one temporary conversation in which two nodes compare their stored information and exchange missing records. A partition is a network split that prevents some running nodes from communicating. Partition recovery is the work that reconnects these nodes and brings their stored information together after communication becomes possible again.

This section describes the serving side of the conversation and the selection of peers to contact. For example, B must answer A's requests while also asking A for records B lacks. If the connection breaks, both sides close it and release their temporary read views; already committed records remain useful. Occasional contact with configured seeds or retained peers makes reconnection possible even when the local membership table considers those peers unavailable.

Stream state machine and resource ownership

A stream is an ordered connection carrying successive messages. A state machine is a set of allowed steps and responses as a conversation progresses. Resource ownership identifies which session is responsible for closing its connection and releasing its temporary read views.

Pseudocode

```
serveReconcile(session):
  require authenticatedAndAuthorized(session.peer)
  require sessionSlotAvailable() or return BUSY
  acquireSessionSlot()
  try:
    verifyNegotiationAndDeadlines()
    snapshots = pinImmutableSnapshots()
    advertiseTokensAndSortedInventoryPages(snapshots)
    while session.active and beforeAbsoluteDeadline():
      frame = receiveBoundedFrame()
      switch frame.kind:
        GET_RECORD:
          require tokenBelongsToSession(frame.token)
          r = snapshots.lookupExact(frame.token, frame.key, frame.stamp)
          require r != NONE and hash(r) == frame.expectedHash
          sendBoundedRecordChunks(r)
        MEMBER_PAGE: validateAndPostMerges(frame)
        INVENTORY_PAGE: routeToLocalFetchDirection(frame)
        RECORD_CHUNK: routeToBoundedRecordReassembler(frame)
        COMPLETE:    markRemoteDirectionComplete()
        otherwise:  reject PROTOCOL_ERROR
      driveLocalInventoryAndFetchDirectionWithoutBlocking()
    require bothDirectionsComplete()
  finally:
    unpinSnapshots(); closeStream(); releaseSessionSlot()

chooseRepairPeer():
  repairSelectionCounter += 1
  if repairSelectionCounter modulo recoveryCadence == 0 or
     not availableActivePeer():
    return roundRobinAuthorizedSeedOrRetainedNonLEFTPeer()
  return randomActivePeer()
```

Each endpoint must serve reads while driving its own fetches. Use a multiplexed stream or separate bounded reader and writer tasks. The fetch direction implements the inventory loop in section 12; its record reassembler validates total size, request ID, chunk sequence and final hash before posting a merge. Bootstrap reuses its authenticated JOIN stream. Limit requests in flight, page entries, total snapshot bytes and session duration. Cancel and release every owned resource on failure.

Concurrent writes after a snapshot opens may miss that session. They remain in live state for later gossip or repair. A failed session may have applied some records; no rollback is required because merges are monotonic and retryable. A successful session means both snapshots were reconciled, not that all nodes are globally current.

Healing a partition

Healing means restoring communication across a network split. Nodes then need to find each other again, correct stale availability reports and exchange any stored updates that were missed while the split lasted.

Peers on opposite sides can falsely mark each other DEAD. An occasional repair attempt to a seed or retained non-LEFT endpoint allows reconnection even when active-peer sets are empty. Exchanged reports about a live self instance trigger a newer owner announcement. A returning process uses a fresh instance and bootstrap. No repair path changes leader ownership or bypasses a separate consensus quorum.

14 Leave retention and failure cleanup

Leave is the procedure by which a process deliberately announces that it is departing. Retention means continuing to keep records that are no longer active but are still needed to interpret delayed information. Cleanup means releasing resources the local program owns, such as open connections, timers and temporary read views.

For example, A stops accepting new work, saves and spreads its final LEFT report, then closes its resources. Other replicas retain that report so a delayed ALIVE report cannot revive A's old process identity. Similarly, a deletion marker is retained so an offline replica cannot restore an older value. This section specifies orderly departure, storage-failure shutdown and why these retained records cannot be discarded merely because a local waiting period ended.

Graceful departure and process identity

Graceful means the process has time to perform its shutdown procedure. DRAINING rejects new application work while the node prepares to stop. An owner-signed LEFT report says this process lifetime has deliberately ended. A restart uses a fresh instance identifier, so late messages about the old lifetime cannot control the new one.

A grace deadline bounds how long the process spends attempting to spread its final report. It is a local time limit, not an acknowledgment from the entire cluster. A peer that misses LEFT may discover the disappearance later through probing.

Follow leave and stop

1. leave disables the application API and cancels regular background schedules. It advances the owner incarnation, persists LEFT and queues the final report.

2. Until the grace deadline, it makes bounded attempts to send that report. It then calls stop whether or not all peers were contacted.

3. stop invalidates timer tokens, cancels workers and waits within a shutdown deadline, closes streams and snapshots, then releases listeners and storage. Owned resources must have a cleanup path even when a worker fails.

4. enterStorageFailure stops the node when required durable state cannot be committed. The node must not acknowledge or advertise a change that its storage did not retain.

Why retention is necessary

Retention means continuing to store information after it is no longer returned as a live value or active member. Garbage collection is the later removal of that retained information. A tombstone suppresses older values; a terminal member record suppresses resurrection of an old instance. Removing these locally can allow an offline replica to reintroduce stale information.

Small example

A deletes colour while C is offline. If A removes the deletion marker too soon, C can return with its old red value and make it look new to a node that forgot the deletion. The baseline keeps the marker. Finite retention needs an additional proof or epoch policy that excludes stale replicas until they bootstrap.

What shutdown guarantees

An acknowledged write is committed locally, but may still be the only copy. Destroying that disk before replication can lose it. Cleanup prevents leaked resources; it does not create a replication quorum or cluster-wide durability guarantee.

Pseudocode

```
leave(graceDeadline):
  require mode == RUNNING
  mode = DRAINING
  disableLocalApplicationAPI()
  cancelPeriodic(PROBE_TICK, GOSSIP_TICK, REPAIR_TICK)
  selfOwner = signOwner(self.instance, self.incarnation + 1, metadata)
  final = ownerRecord(selfOwner, LEFT)
  persistMemberBeforeApplying(final) or enterStorageFailure()
  members[self.instance] = final
  enqueue(final)
  while now() < graceDeadline and sendBudgetRemains():
    sendFinalToBoundedPeerSample(final)
    flushAvailableControlTraffic()
  stop()

stop():
  mode = STOPPED
  invalidateAllTimerTokens()
  cancelAndJoinWorkersWithinShutdownDeadline()
  closeAllStreamsAndReleaseSnapshots()
  closeListenersAndStorage()
  clearVolatileRequestsAndQueues()

onRelayTimeout(token):
  if pendingRelays.contains(token): releaseRelay(token)

enterStorageFailure():
  disableLocalApplicationAPI()
  Metrics.emit(STORAGE_FAILURE)
  stop()                            # do not advertise uncommitted state
```

Graceful leave improves propagation but cannot prove every peer learned LEFT. Restart creates a new instance. Any write acknowledged before leave is locally durable; graceful shutdown does not establish remote durability. A crash can lose the only surviving copy if that node and its disk are lost before replication.

Retention is part of correctness

Retention is the continued storage of a deletion or departure record after it stops representing live data or an active member. Correctness depends on keeping enough of this information to reject delayed older records.

The baseline retains application tombstones and terminal member records. Deleting either on a local timer can resurrect old values or old instances. Cassandra documents this risk for deletion markers [6]. The baseline therefore does not provide a time-only purge operation.

For finite retention, add a separate epoch or stable-watermark mechanism that proves every admitted replica observed the deletion, and force replicas outside that proof to discard obsolete state and bootstrap before serving. Garbage collection must cover offline backups and retired identities too. A local timeout or one successful pairwise repair is insufficient.

15 State transitions and worked examples

A state transition is a change from one recorded condition to another, such as changing a member from ALIVE to SUSPECT. A worked example follows specific messages and records through the rules so that the consequences can be seen in order.

This section follows a stored update, a suspected failure, competing edits, a deletion and a healed network split. In the diagram, i means one incarnation number, which versions a process's own announcements; i plus 1 means a newer announcement. Read each arrow as a permitted change and the accompanying text as the reason that change occurs.

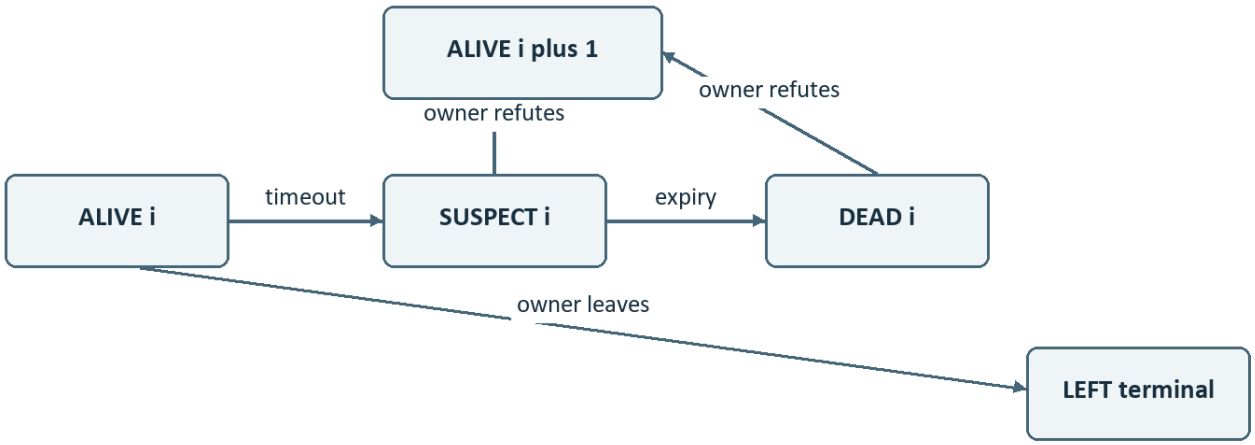


Figure 5 Suspicion can be refuted only with a newer owner incarnation. LEFT ends one process lifetime; rejoining uses a new instance.

A write reaches another replica

A commits key temperature at stamp (8,A). Its rumor reaches B, which stores the record and queues it for further dissemination. A duplicate is ignored by the value merge. C misses every rumor. During a later repair with B, C advertises no record for that key and fetches stamp (8,A). All three now return the same locally stored value, assuming no newer write.

A slow process avoids a false death declaration

A fails to obtain B's ACK directly or through helpers and reports SUSPECT at incarnation 4. B receives the signed report and publishes ALIVE at incarnation 5. A stores that newer announcement and cancels its suspicion token. If the old timer event is already queued, its token check prevents a DEAD transition. A delayed ALIVE at incarnation 4 cannot undo SUSPECT at 4.

Concurrent edits and a deletion

A and B are disconnected. A writes value red at (12,A); B deletes the same key at (12,B). Suppose B sorts after A. After repair, the deletion wins everywhere. If A later observes (12,B) and intentionally writes blue, its new stamp is at least (13,A), so blue wins. Tombstones prevent old red from returning; they do not prohibit future explicit writes.

A partition heals

A and B mark each other DEAD at incarnation 6. A later reconnects through its retained peer list. Each sees a DEAD report about itself, advances to incarnation 7 and disseminates ALIVE. Their data snapshots merge independently. If B had stopped and restarted, its new identifier B2 would join while the old B record remained terminal or subject to failure detection.

16 Guarantees and limits

A guarantee is a behavior the design promises when its stated assumptions hold. A limit identifies a behavior it does not establish. In a distributed system, those assumptions include whether communication eventually succeeds, whether nodes have enough storage, and whether admitted participants follow the protocol.

For example, nodes that eventually receive the same valid writes choose the same winning value because they use the same comparison rule. That is convergence. It does not mean every node sees a write immediately or that a failed probe proves a crash. This section separates those claims and explains why agreeing on exclusive authority requires a separate coordination protocol.

Property	Reference guarantee	Necessary qualification
Duplicate handling	Repeated records do not change the result	Callbacks are only local change notifications
Conflict convergence	Same valid records produce the same winning value	All nodes use the identical stamp and merge policy
Eventual repair	Retained updates can cross after connectivity returns	Fair scheduling successful delivery and enough capacity
Failure detection	Missing ACKs lead to suspicion then local DEAD	A partition or pause can cause false positives
Local write durability	Acknowledgment follows a durable local transaction	No replication quorum or disk-loss tolerance implied
Deletion safety	Retained tombstones suppress older records	Intentional newer writes can recreate the key
Ordering	Per-key deterministic stamp order	No total event order or atomic multi-key state
Membership	Eventually reconciled local views	No authoritative simultaneous cluster view

Reasoning about convergence

Convergence means replicas settle on the same stored result after the relevant information reaches them. The argument below explains which comparison rules and communication assumptions make that happen.

For a fixed key, selecting the maximum valid stamp is independent of delivery order and duplicate count. Once all communicating replicas have received the same finite set of writes, they retain the same winner. Periodic repair supplies updates that bounded rumors miss. The argument assumes valid unique writer stamps, retained state, eventual communication and no permanent overload.

Membership differs from application state because suspicions are generated locally and owner refutations generate new versions. Under stable connectivity and responsive processing, reports and refutations can settle to matching views. In a fully asynchronous system, no timeout can distinguish a failed process from an arbitrarily slow one.

Consensus boundary

Consensus is a separate protocol for reaching an agreed decision under explicit failure rules. The boundary here is the point where spreading and merging information is insufficient to decide who has exclusive authority.

Do not use gossip membership alone to decide a unique leader, transfer ownership, grant an exclusive lease, remove a voter, or authorize duplicate-sensitive work. Those decisions need a coordination mechanism with its own safety rules. In a Raft-based system, membership gossip may offer discovery hints; Raft configuration and quorum rules remain authoritative.

17 Cost tuning and observability

Cost is the network traffic, processor work, memory and storage the protocol consumes. Tuning means adjusting its settings to trade these resources against response time. Observability means collecting measurements and events that help an operator understand how the running system is behaving.

For example, increasing the number of peers contacted on each gossip action can spread an update sooner, but sends more messages. A longer suspicion period gives a busy peer more time to refute suspicion, but delays removing a crashed peer. This section estimates resource use, explains these tradeoffs, and names measurements that reveal delayed processing, dropped work or unsuccessful repair.

A budget that can be measured

Let N be active peers, f the gossip fanout, k the helper fanout, g the gossip period, p the probe period, B the datagram limit, S the retained state size and r the repair interval. Per-node scheduled gossip sends are at most f/g per second, contributing at most fB/g bytes per second. Probe traffic adds a bounded constant number of messages per probe when k is fixed; received helper work depends on cluster scheduling and rate limits.

Ideal epidemic spreading often completes in logarithmically many rounds under suitable mixing and delivery assumptions [2]. That is an analytical model, not a latency service level. At fixed per-packet budgets, high churn and overloaded queues can dominate propagation time.

The baseline repair scans O(N + K) inventory entries per session for K retained application keys, even when few values differ. It transfers changed record payloads in addition to that scan. Retained historical instances increase membership size. Full application replication consumes O(S) storage per node and roughly O(NS) aggregate storage. No constant-cost claim applies to snapshot repair or retained history.

Tradeoffs

Increase	Benefit	Cost
Gossip fanout or frequency	Faster repeated spreading	More packets and contention
Indirect helper count	More alternate paths to a target	More helper work
Suspicion timeout	More opportunity to refute	Slower removal of crashed peers
Repair frequency	Shorter time to recover missed rumors	More inventory I O and network work
Queue capacity	More pending change coverage	More memory and possibly older backlog

Metrics and diagnostic events

A metric is a numerical measurement, such as a count of failed probes or the time a repair took. A diagnostic event records something that happened, such as a refutation or a storage failure. Together they help explain the system's behavior.

Record direct and indirect probe latency, failed probes, suspects, refutations, DEAD and LEFT transitions, event-loop delay, control queue delay, rumor bytes and evictions, repair duration and failures, snapshot age, retained bytes, invalid signatures and local write failures. Break down by message kind and peer without logging application values or secrets.

Optional Lifeguard extensions adapt probing to local health, shorten suspicion after independent confirmations and prioritize notifying the suspected peer [7]. The baseline already sends an urgent notice, but does not claim to implement the complete Lifeguard algorithm. Add adaptive timers only with corresponding state and validation scenarios.

18 Behavioral validation plan

Behavioral validation checks what an implementation does when given particular inputs and failures. A validation plan names the situations to exercise and the observable result expected in each. The purpose is to check the protocol's decisions, not merely that its functions were called.

For example, deliberately lose B's direct reply but allow a helper to deliver B's answer. The expected result is a completed probe without a new suspicion. Other scenarios deliver old messages, interrupt repair, restart processes and fill queues. The table below can guide tests of an implementation; it is not a claim that this explanatory pseudocode has been executed or verified.

Use repeatable simulated message schedules, real encoders and decoders, and a purpose-built network fixture that can lose or delay packets. Test network adapters with real local endpoints. These are future implementation checks; no mocking framework is needed.

Scenario	Observable acceptance condition
Bootstrap and seed failure	Retries another seed; never forms an accidental new cluster
Duplicate and reversed delivery	Every key retains the same maximum stamp
Direct timeout indirect success	Probe completes without creating a new suspicion
Forged relay or wrong target	ACK is rejected; unrelated probe remains pending
Refutation with queued expiry	New ALIVE survives the obsolete timer event
Repeated suspicion reports	Duplicate reports never extend the initial deadline
Crash and fresh restart	Old messages affect old instance only; new instance bootstraps
Concurrent update and delete	All replicas choose the declared stamp winner
All rumors dropped	Retained state converges through periodic repair
No active peers after partition	Retained-peer or seed contact enables repair and refutation
Write during repair snapshot	Next session or rumor transfers the newer record
Broken stream and cancellation	No leaked sessions snapshots workers or request slots
Queue and packet saturation	Control traffic remains bounded; evicted data remains repairable
Disk failure before acknowledgment	No LOCAL_COMMITTED result is emitted
Invalid and incompatible traffic	No unauthorized mutation; bounded decode and work
Logical counter near maximum	Writes fail before overflow; no stamp reuse
LEFT and delayed ALIVE	Old instance stays LEFT; new instance can join

Properties worth checking across generated schedules

A property is a rule expected to remain true through many actions. A generated schedule is a simulated ordering of message deliveries, timer expiries and failures. Trying different schedules exposes errors that a single successful exchange can miss.

Value stamps never decrease. Duplicate merges are idempotent. A timer can act only on its matching incarnation and token. Every admitted request has a terminal completion or timeout path. After writes stop and the simulated network becomes connected and fair, retained values converge. Run real multi-process partition and pause tests before claiming product compatibility or operational readiness.

19 Sources and further design choices

Sources are the research papers, official implementation material and documentation used to establish the underlying ideas. A design choice is a particular rule selected for this reference from among possible alternatives. Keeping the distinction clear helps readers tell an established technique from a decision they may change.

For example, SWIM informs the separation of peer probing and membership spreading. The exact record fields, module boundaries and application-value comparison in this document form a separate reference design. The links below let readers inspect the primary material; they do not imply that these pseudocode messages are compatible with the cited products.

The sources below were researched on the web on 17 October 2026. Bracketed references identify the relevant source. Pseudocode and baseline contracts are original synthesis.

[1] Alan Demers and colleagues 1987

Epidemic algorithms for replicated database maintenance. PODC. Foundational distinction between anti entropy and rumor spreading; treatment of deletion information.

Open primary source

[2] Abhinandan Das Indranil Gupta and Ashish Motivala 2002

SWIM Scalable Weakly consistent Infection style Process Group Membership Protocol. DSN. Randomized probing, indirect checks, suspicion and membership dissemination.

Open primary source

[3] HashiCorp memberlist

Official repository and implementation overview. A practical SWIM derived membership library with additional convergence and local-health behavior.

Open primary source

[4] HashiCorp memberlist broadcast queue

queue.go. Primary implementation reference for bounded retransmission and packing broadcasts to a byte limit. The repository mainline is mutable.

Open primary source

[5] Giuseppe DeCandia and colleagues 2007

Dynamo Amazon's Highly Available Key value Store. SOSP. Primary example of object versioning and Merkle tree based replica synchronization.

Open primary source

[6] Apache Cassandra documentation

Tombstones. Explains deletion markers, grace periods and the danger of old data returning after premature marker removal.

Open primary source

[7] Armon Dadgar James Phillips and Jon Currey 2017

Lifeguard Local Health Awareness for More Accurate Failure Detection. Describes local health aware probes, dynamic suspicion and the buddy system.

Open primary source

[8] HashiCorp memberlist security model

SECURITY.md. Describes the trust and security boundaries of shared-key gossip, including lack of per-node authorization and Byzantine tolerance.

Open primary source

20 Security and extension contracts

Security rules decide who can participate, whether a message was authored by the claimed participant, and what that participant is permitted to change. Authentication establishes identity or authorship; authorization grants permission; encryption protects message contents from observation. An extension is an added or replacement capability. Its contract states the behavior the other modules may rely on.

For example, receiving a correctly signed value does not automatically authorize its writer to change every key. The writer must also have permission for the relevant namespace, a named group of application keys. Replacing the value comparison with a different conflict policy must also update how queues and repair compare values. This section specifies those boundaries and identifies the extra rules required by optional capabilities.

Security is an explicit boundary

A security boundary states which participants and actions the system trusts. Checking a signature establishes an author; admission and authorization decide whether that author may participate and make the requested change.

The reference requires encrypted authenticated channels and signatures on relayed records. Owner and reporter signatures bind records to admitted identities; permission checks restrict endpoints and namespaces. An implementation must supply these checks.

memberlist's security model explains that possession of a group key defines its trust boundary and does not establish individual authorization or Byzantine tolerance [8]. A malicious admitted reporter can still lie about reachability, and an authorized writer can still issue damaging values. Signatures establish authorship; they do not establish honesty.

Pseudocode

```
validMember(r):
  require verifyOwnerSignature(r.owner)
  require admissionPolicyAllows(r.owner)
  require recordBoundsAndClusterMatch(r)
  if r.status in {ALIVE, LEFT}:
    require r.reporter == r.owner.instance
    require verifyOwnerReportSignature(r)
  else:
    require admittedReporter(r.reporter)
    require verifyReporterSignature(r)
  return true

validValue(r):
  require recordBoundsAndClusterMatch(r)
  require verifyWriterSignature(r)
  require principalAllowedForNamespace(r.writerPrincipal)
  require instanceBoundToPrincipal(r.stamp.writer, r.writerPrincipal)
  require r.deleted implies r.value == NONE
  require r.stamp.counter is representable
  return true
```

Replacement points

A replacement point is a module that can be exchanged for a different implementation or policy. The table names the extra agreement needed so the remaining modules still interpret its results correctly.

Need	Replace or extend	Additional contract
Preserve concurrent edits	Application merge module	CRDT or explicit causal conflict policy
Reduce full scan cost	Repair inventory module	Canonical Merkle partitioning and snapshots
Very large peer groups	Membership and peer sampler	Partial views with connectivity repair
Durable event delivery	Application API and storage	Replay log offsets acknowledgments retention
Exclusive ownership	Coordination layer	Consensus or leases with fencing
Finite tombstone storage	Admission and retention	Safe watermark or epoch and forced bootstrap

These extensions change guarantees. Specify and validate them as independent modules rather than assuming that increasing gossip fanout supplies the missing safety property.

Additional concepts used in the reference

A concept glossary is a quick explanation of terms used elsewhere in the document. This glossary covers additional ideas used to discuss correctness, security and optional changes. It can be used for lookup after reading the functional description at the beginning of each section.

Concept	Meaning in this document
Consensus	A protocol for reaching an agreed decision under explicit failure assumptions. Gossip spreading and matching local values alone do not implement it.
Quorum	A set of participants large enough to satisfy a coordination protocol's voting or acknowledgment rule. The local write path in this reference does not wait for one.
Lease and fencing	A lease grants authority for a bounded interval. Fencing uses an ordered authority token that a resource checks to reject an obsolete holder. Gossip reachability alone grants neither.
Exactly once execution	A requirement that a business effect occurs one time despite retries or failures. Duplicate-safe state merging does not establish this for callbacks or external actions.
Causal order	The before-and-after relationship created when an operation observes an earlier operation. Disconnected concurrent writes need a separate conflict rule.
Total order	A comparison rule that orders every distinct pair. Value stamps provide a per-key winner; they do not create a cluster-wide ordered event log.
CRDT	Conflict-free replicated data type: a data structure designed with compatible merge or update rules so replicas can converge. Its exact semantics must match the application.
Namespace	A named group of application keys that share a replication and authorization policy. The baseline replicates one configured namespace to every admitted node.
Serialization	Turning structured fields into bytes for storage or transport. Parsing performs the reverse. Canonical serialization uses one fixed representation for signing and hashing.
Authentication and authorization	Authentication establishes an identity or authorship. Authorization decides what that identity is permitted to do. Encryption protects the content from observation.
Byzantine behavior	Arbitrary or dishonest behavior by a participant, such as lying about another node. The reference assumes cooperative admitted nodes and does not tolerate all such behavior.
Invariant	A property that must remain true after every permitted operation. An example is that a retained application stamp never decreases.
O notation and logarithms	O(N) describes cost that grows at most proportionally with N in the stated model. O(log N) describes slower logarithmic growth. ln uses base e; log2 uses base 2. Neither predicts seconds or bytes without the model and constants.

Keep the responsibilities separate

A probe answers whether a reply arrived in time. A membership merge selects a locally retained status. A value merge selects a stored value. Repair supplies missing records. A separate coordination layer must decide exclusive authority. Reading each algorithm against that responsibility helps avoid assigning it guarantees it does not provide.